

3.Complex Data Types: Objects

Detailed Explanation:

- **Objects** in JavaScript are collections of key-value pairs. They allow storing multiple values of different types, where each value is associated with a unique key. Objects can be created using the **object literal** syntax {}.
- **Dot notation** is used to access or modify an object's properties.
- You can also add or delete properties dynamically:
 - Use delete to remove a property from an object.

Code Snippet:

javascript

Copy code

// Creating an object

```
let user = {  
  name: "John",  
  age: 30,  
  email: "john@example.com"  
};
```

// Accessing object properties

```
console.log(user.name); // Output: John
```

// Modifying object properties

```
user.age = 31;  
console.log(user.age); // Output: 31
```

// Adding new properties

```
user.phone = "123-456-7890";  
console.log(user.phone); // Output: 123-456-7890
```

```
// Deleting properties  
delete user.email;  
console.log(user.email); // Output: undefined
```

In-depth Questions:

1. **What is the purpose of an object in JavaScript?**
 - **Answer:** Objects allow storing multiple values (key-value pairs) in a single variable, making it easier to manage related data (e.g., user information).
2. **How can you add a new property to an object dynamically?**
 - **Answer:** By using dot notation, you can assign a value to a new property, and it will be added dynamically to the object (e.g., user.phone = "123-456-7890";).

Practice Question:

- **What does the delete keyword do in JavaScript?**
 - **Answer:** It removes a property from an object.
-

2. Complex Data Types: Arrays

Detailed Explanation:

- **Arrays** are used to store collections of data. They are ordered lists of values that can be of any type. Elements are accessed using **index notation**, where the first element is at index 0.
- You can store elements of different types, including nested arrays (multidimensional arrays).

Code Snippet:

```
javascript  
Copy code  
// Array with different data types  
let arr = ["JavaScript", 42, true, [1, 2, 3]];  
  
// Accessing array elements  
console.log(arr[0]); // Output: JavaScript  
console.log(arr[3][1]); // Output: 2  
  
// Modifying array elements
```

```
arr[1] = 50;
```

```
console.log(arr[1]); // Output: 50
```

In-depth Questions:

1. **How do you access an element from a nested array in JavaScript?**

- **Answer:** You use multiple square brackets to access elements from a nested array (e.g., `arr[3][1]` to access the second element of the nested array).

2. **What is the first index of an array in JavaScript?**

- **Answer:** The first index of an array is 0.

Practice Question:

- **True or False:** You can store values of different data types in a JavaScript array.
 - **Answer:** True.
-

3. Array Methods

3.1 length Property

- **length** gives the number of elements in an array, including empty spaces.

Code Snippet:

```
javascript
```

Copy code

```
let names = ["Olivia", "Emma", "Mateo", "Samuel"];
```

```
console.log(names.length); // Output: 4
```

3.2 indexOf Method

- **indexOf** searches for a value in an array and returns its index. If the value is not found, it returns -1.

Code Snippet:

```
javascript
```

Copy code

```
let names = ["Olivia", "Emma", "Mateo", "Samuel"];
```

```
console.log(names.indexOf("Mateo")); // Output: 2
```

```
console.log(names.indexOf("Victor")); // Output: -1
```

3.3 push and unshift Methods

- **push** adds an element to the end of an array, and **unshift** adds an element to the beginning.

Code Snippet:

javascript

Copy code

```
let names = ["Olivia", "Emma"];
names.push("Mateo");
console.log(names); // Output: ["Olivia", "Emma", "Mateo"]

names.unshift("Samuel");
console.log(names); // Output: ["Samuel", "Olivia", "Emma", "Mateo"]
```

3.4 pop and shift Methods

- **pop** removes and returns the last element, while **shift** removes and returns the first element.

Code Snippet:

javascript

Copy code

```
let names = ["Olivia", "Emma", "Mateo"];
let lastElement = names.pop();
console.log(lastElement); // Output: Mateo

let firstElement = names.shift();
console.log(firstElement); // Output: Olivia
```

3.5 reverse Method

- **reverse** reverses the order of elements in an array.

Code Snippet:

javascript

Copy code

```
let names = ["Olivia", "Emma", "Mateo"];
names.reverse();
```

```
console.log(names); // Output: ["Mateo", "Emma", "Olivia"]
```

3.6 slice Method

- **slice** creates a new array from a portion of an existing array.

Code Snippet:

javascript

Copy code

```
let names = ["Olivia", "Emma", "Mateo", "Samuel"];  
let slicedNames = names.slice(1, 3);  
console.log(slicedNames); // Output: ["Emma", "Mateo"]
```

3.7 concat Method

- **concat** combines two arrays into a new one without changing the original arrays.

Code Snippet:

javascript

Copy code

```
let names = ["Olivia", "Emma"];  
let otherNames = ["Mateo", "Samuel"];  
let allNames = names.concat(otherNames);  
console.log(allNames); // Output: ["Olivia", "Emma", "Mateo", "Samuel"]
```

In-depth Questions:

1. **What does the push method do in JavaScript?**
 - **Answer:** The push method adds an element to the end of an array.
2. **What is the difference between pop and shift?**
 - **Answer:** pop removes the last element from an array, while shift removes the first element.

Practice Question:

- **What will the following code output?**

javascript

Copy code

```
let names = ["Olivia", "Emma", "Mateo"];
```

```
let newNames = names.slice(1);
```

```
console.log(newNames);
```

- **Answer:** ["Emma", "Mateo"].
-

4. Tasks from the File

- **Task 1:** Create an object that describes a train ticket with from, to, and price properties.

javascript

Copy code

```
let ticket = {  
  from: "Central Station",  
  to: "North Station",  
  price: 25  
};  
  
console.log(`${ticket.from} -> ${ticket.to}, Price: ${ticket.price}`);
```

- **Task 2:** Create an empty object and add firstName and lastName properties.

javascript

Copy code

```
let person = {};  
  
person.firstName = "Stavan";  
person.lastName = "Kodolika";  
  
console.log(`${person.firstName} ${person.lastName}`);
```

- **Task 3:** Create an array of book objects with properties title, author, and pages.

javascript

Copy code

```
let books = [  
  { title: "Speaking JavaScript", author: "Axel Rauschmayer", pages: 460 },  
  { title: "Programming JavaScript Applications", author: "Eric Elliott", pages: 254 },  
  { title: "Understanding ECMAScript 6", author: "Nicholas C. Zakas", pages: 352 }  
];
```

- **Task 4:** Add a new book to the collection using push.

javascript

Copy code

```
books.push({ title: "Learning JavaScript Design Patterns", author: "Addy Osmani", pages: 254 });  
console.log(books.length); // Output: 4
```

- **Task 5:** Use slice to copy the last two books to a new array.

javascript

Copy code

```
let lastTwoBooks = books.slice(-2);  
console.log(lastTwoBooks);
```

- **Task 6:** Remove the first book from the collection using shift.

javascript

Copy code

```
books.shift();  
console.log(books.length); // Output: 3
```

- **Task 7:** Display the total number of pages for all books.

javascript

Copy code

```
let totalPages = books.reduce((sum, book) => sum + book.pages, 0);  
console.log(`Total pages: ${totalPages}`);
```